Vulnerability Assessment Report

Imagur - Full-Scope Vulnerability Assessment (October 2026)

Imagur
132 Morphett Rd
Glengowrie SA 5044, Australia

Flinders University
Sturt Road, Bedford Park SA 5042

October 5, 2026

Report Version: 1.0

CONFIDENTIAL

Table of Contents

List of Figures 4

1 Document Control 5

1.1 Team 5

1.2 List of Changes 5

2 Executive Summary 6

2.1 Overview 6

2.2 Identified Vulnerabilities 6

3 Methodology 8

3.1 Objective 8

3.2 Scope 8

3.3 User Accounts and Permissions 9

4 Findings 10

C1: Publicly Exposed AWS Root Account Credentials in Client-Side JavaScript 10

C2: Public S3 Bucket - All User Images (Including "Private") Are World-Readable and Enumerable 12

H1: Mass Assignment / IDOR on Image Upload (Attacker Uploads Images to Any Victim Account) 14

H2: Broken Access Control on Image Visibility Toggle (IDOR - Attacker Makes a Victim's Private Image Public) 16

H3: Production Application Also Exposed Over Plaintext HTTP on Port 8080 18

M1: No Rate Limiting on /login (Enables Credential Brute Force) 20

M2: PostgreSQL Database Server Exposed to the Public Internet 21

M3: Private Images Expose Long-Lived Pre-signed URLs That Bypass "Private" Status 23

M4: Weak Password Policy - Single-Character Passwords Accepted 25

M5: Open DNS Resolver on Port 53 (Potential DDoS Amplification Reflector) 27

M6: Dangling / Misconfigured CloudFront Alias on resources.imagesharer.xyz 29
M7: Unhandled Server Error on Empty user_id Parameter (Denial via Malformed Input) 31
M8: User-Enumeration via Registration Endpoint 32
M9: Missing HTTP Security Headers (HSTS, CSP) and Email Anti-Spoofing (SPF, DMARC) 33
I1: SSH (OpenSSH 10.0p2) Exposed to the Public Internet 35
5 Disclaimer 36
A Appendix 37
A.1 Static Appendix Section 37
A.2 Methodology 37
A.3 Remediation Summary (Executive) 37

List of Figures

Figure 1 - Distribution of identified vulnerabilities 7

1 Document Control

1.1 Team

Contact	Details	Role
ClawSec	Autonomous agent	Pentester

1.2 List of Changes

Version	Description	Date
1.0	Initial release.	Oct 5, 2026

2 Executive Summary

2.1 Overview

Imagur engaged ClawSec to perform a one-day, full-scope vulnerability assessment of the "Imager" image-sharing platform on 5 October 2026. The assessment covered the public website (imagesharer.xyz), its underlying OVH-hosted server, and the associated AWS infrastructure that was proven to belong to the business.

In short: the service is at **immediate, severe risk** on multiple independent fronts. Fifteen issues were confirmed, of which **two are critical and four are high**.

The single most urgent problem is a **live AWS root-account access key published on the public website** inside a JavaScript file. Any visitor can read it; our verification call confirmed it grants unrestricted administrative control of the AWS account that runs the platform. In parallel, the **S3 bucket that stores every uploaded image is open to the entire internet** — anyone can list every file and download any image, including pictures users explicitly marked as "Private". The platform's privacy promise is therefore not met today.

Within the web application itself, two broken access-control defects allow any logged-in member to (a) upload images onto another member's account and (b) flip another member's private image to public. Combined, these let an attacker plant content on a victim's account and then expose it publicly. The database server (PostgreSQL) is reachable from the open internet, the application is additionally served in cleartext on port 8080, and the server is acting as an open DNS resolver that could be used by third parties for amplification attacks. A handful of smaller issues — weak password policy enforced only in the browser, no login rate limiting, email-enumeration on registration, missing security headers, and missing email anti-spoofing records — further weaken the overall security posture.

Recommended priority: rotate the AWS root key and lock down the S3 bucket within hours; fix the two authorization defects in the upload and visibility endpoints before any additional user traffic is onboarded; firewall PostgreSQL and port 8080; disable the open DNS resolver. The smaller hygiene issues can be scheduled across the following sprint.

Fixing the critical and high items is low-cost and well-understood work. Doing so would move Imager from "all user data effectively public" to a defensible baseline suitable for a growing user base.

2.2 Identified Vulnerabilities

#	CVSS	Description	Page
C1	10.0	Publicly Exposed AWS Root Account Credentials in Client-Side JavaScript	10
C2	9.3	Public S3 Bucket - All User Images (Including "Private") Are World-Readable and Enumerable	12
H1	7.6	Mass Assignment / IDOR on Image Upload (Attacker Uploads Images to Any Victim Account)	14
H2	7.1	Broken Access Control on Image Visibility Toggle (IDOR - Attacker Makes a Victim's Private Image Public)	16

#	CVSS	Description	Page
H3	7.1	Production Application Also Exposed Over Plaintext HTTP on Port 8080	18
M1	6.5	No Rate Limiting on /login (Enables Credential Brute Force)	20
M2	6.5	PostgreSQL Database Server Exposed to the Public Internet	21
M3	6.5	Private Images Expose Long-Lived Pre-signed URLs That Bypass "Private" Status	23
M4	6.5	Weak Password Policy - Single-Character Passwords Accepted	25
M5	5.8	Open DNS Resolver on Port 53 (Potential DDoS Amplification Reflector)	27
M6	5.4	Dangling / Misconfigured CloudFront Alias on resources.imagesharer.xyz	29
M7	5.4	Unhandled Server Error on Empty user_id Parameter (Denial via Malformed Input)	31
M8	5.3	User-Enumeration via Registration Endpoint	32
M9	4.2	Missing HTTP Security Headers (HSTS, CSP) and Email Anti-Spoofing (SPF, DMARC)	33
I1	0.0	SSH (OpenSSH 10.0p2) Exposed to the Public Internet	35

Vulnerability Overview

In the course of this penetration test **2 Critical**, **3 High**, **9 Medium** and **1 Info** vulnerabilities were identified:

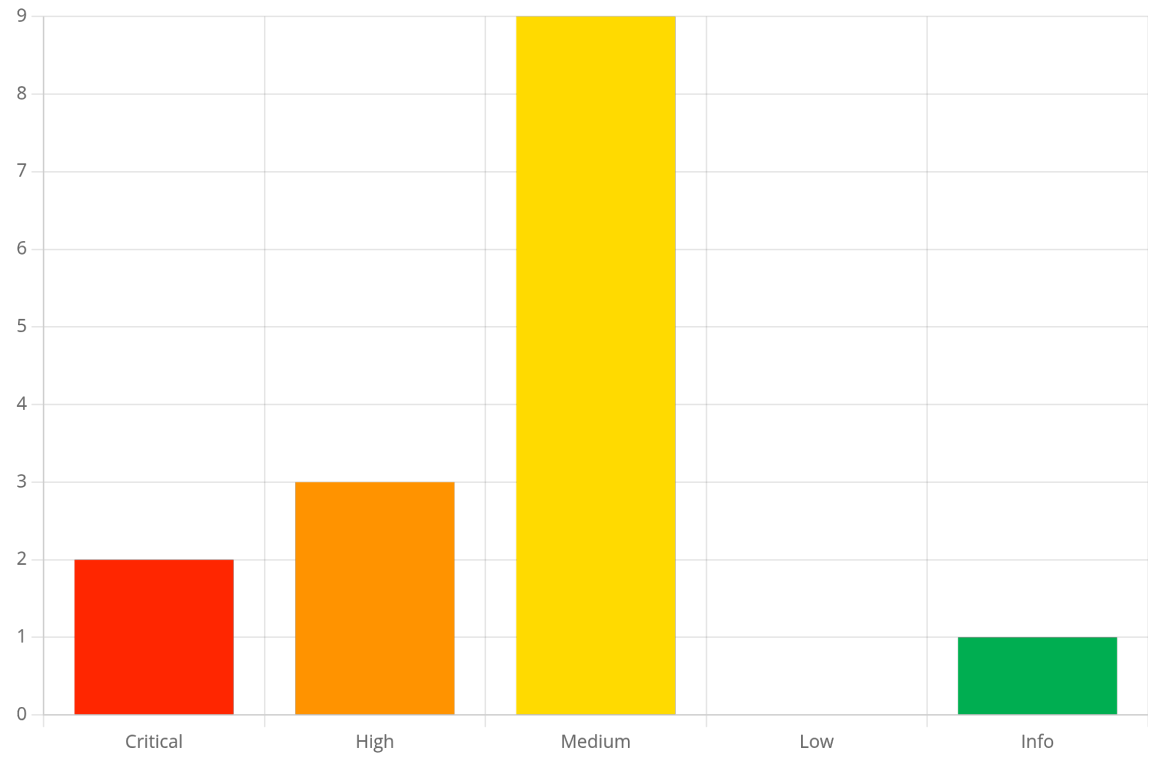


Figure 1 - Distribution of identified vulnerabilities

3 Methodology

This entire vulnerability assessment process was completed using agentic AI and existing LLM models. The purpose of such assessments is to automatically

3.1 Objective

The entire objective of this research is to evaluate the effectiveness of autonomously evaluate security posture for any business available on Google Map without the needs of technical guidance to determine how feasiable, seamless and cost-affective such approach is.

3.2 Scope

Imagur authorized ClawSec to perform a time-boxed, full-scope vulnerability assessment of the Imager image-sharing platform. Authorization was confirmed on 5 October 2026 via the required `AuthorizedCode` HTTP header exchange on the primary website.

The assessment covered the public web application, the OVH-hosted server that runs it, and the AWS infrastructure proven (by evidence of ownership) to belong to the business.

System	Description
imagesharer.xyz	Primary production web application (HTTPS)
www.imagesharer.xyz	Alias of the primary web application
resources.imagesharer.xyz	CloudFront-fronted static subdomain
51.161.131.44	OVH Australia VPS hosting the application (ports 80 / 443 / 5432 / 8080 / 22 / 53)
d273uevi0662l0.cloudfront.net	CloudFront distribution — customer configuration only
AWS Account 016170038077	AWS account owning the application's cloud resources (ownership verified via sts:GetCallerIdentity)
s3://filesharer-research (ap-southeast-2)	S3 bucket storing every uploaded image

Out of scope / excluded: `cataierc.com` and its subdomains (co-tenant on the same VPS, no ownership evidence), OVH shared infrastructure beyond the specific VPS, AWS CloudFront shared edge IP ranges, and third-party services (gstatic.com, spaceship.net, letsencrypt.org).

Limitations: No destructive techniques were used — no DDoS, brute force, credential stuffing, password spraying, MFA bypass, social engineering, or exfiltration of sensitive data. PostgreSQL on port 5432 was banner-probed only; no authentication was attempted. The leaked AWS key was used for a single read-only `sts:GetCallerIdentity` call solely to confirm ownership; no further AWS API calls were made with it. The engagement was a single-day window, so areas such as mobile clients, future features, and non-public endpoints behind deeper authentication paths may harbour further issues.

3.3 User Accounts and Permissions

- clawsec-pt-a (test account A, created solely for the engagement)
- clawsec-pt-b (test account B, created solely for the engagement)

4 Findings

C1: Publicly Exposed AWS Root Account Credentials in Client-Side JavaScript	
Score	10.0 (Critical)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H
Target	• https://imagesharer.xyz/js/awsvalid.js • AWS Account 016170038077
References	• https://cwe.mitre.org/data/definitions/798.html • https://cwe.mitre.org/data/definitions/312.html • https://cwe.mitre.org/data/definitions/540.html • https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html#lock-away-credentials

Overview

Imagur's public website serves a JavaScript file that embeds a live AWS access key and matching secret for the AWS root account that owns the application's cloud environment. Any visitor can read the credentials directly from the browser and immediately gain full administrative control of the Imagur AWS account.

Details

The file `https://imagesharer.xyz/js/awsvalid.js` is a public static JavaScript asset. Lines 3-7 contain a commented-out block holding an AWS access key ID, matching secret, and region (ap-southeast-2). The access key ID begins with `AKIA...` and belongs to the root user of AWS account 016170038077.

Ownership was verified with a single read-only `aws sts get-caller-identity` call using the exposed credential, which returned:

```
UserId:  016170038077
Account: 016170038077
Arn:     arn:aws:iam::016170038077:root
```

Because the returned ARN ends in `:root`, this is NOT a scoped IAM user; it is the AWS account root user. Root credentials bypass all IAM policies and permission boundaries and allow an attacker to:

- Read, modify, or delete every resource in the account (S3 data, databases, EC2, IAM users, billing, DNS/Route53, CloudFront, backups).
- Create new IAM users and backdoors.

- Close the AWS account.
- Incur unbounded costs (crypto mining, data egress).
- Change billing/payment information and tax identity.

Business impact. Complete loss of confidentiality, integrity, and availability of all data and services hosted in AWS account 016170038077; likely data breach affecting all users; possible full account takeover, destruction, or financial loss. Internet-wide bots continuously scrape for `AKIA`-prefixed strings, so exposure time (not just presence) drives risk — if the key has been online for any meaningful period it should be treated as already compromised.

Evidence.

- `evidence/p0-awskey/awsvalid.js` (full captured file)
- `evidence/p0-awskey/awsvalid.headers.txt` (HTTP 200, `text/javascript`)
- `evidence/p0-awskey/sts-get-caller-identity.json` (ownership verified)

Note: The secret portion of the credential is deliberately not reproduced in this report.

Recommendation

Immediate (within hours):

1. Log into the AWS Console as root and rotate the root access key: deactivate then delete the exposed key immediately (IAM -> Security credentials -> Access keys).
2. Enable Multi-Factor Authentication (MFA) on the root user if not already enabled.
3. Review CloudTrail event history for AWS account 016170038077 for the last 90 days. Look for any API calls originating from unfamiliar IPs or using the leaked access key ID. Pay special attention to IAM, S3, EC2, and RDS activity.
4. Remove the credential block entirely from `awsvalid.js` and redeploy. Confirm it is no longer served (hard-refresh test).
5. Rotate every other secret that may have been stored in the same account (S3 bucket keys, DB passwords, API tokens), since an attacker with root could have exfiltrated them.

Short term (within days):

6. Do not use the root user for day-to-day operations. Create a dedicated IAM (or IAM Identity Center) user with least-privilege permissions and use that from here on.
7. Enable AWS GuardDuty and billing anomaly alerts to detect reuse of leaked credentials or sudden spend changes.
8. Add a secret-scanning gate to the deployment pipeline (e.g., gitleaks, trufflehog) so no commit containing `AKIA`/`ASIA`/AWS secret patterns can be released.
9. Any client-side code that needs to talk to AWS must use short-lived temporary credentials (Amazon Cognito, STS AssumeRoleWithWebIdentity) — never a long-lived root key.

Long term:

10. Adopt a secret manager (AWS Secrets Manager / HashiCorp Vault) for all backend secrets; never ship secrets in client-side JavaScript regardless of comment status, because every byte sent to the browser is public.

C2: Public S3 Bucket - All User Images (Including "Private") Are World-Readable and Enumerable	
Score	9.3 (Critical)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:L/A:N
Target	• s3://filesharer-research/ (ap-southeast-2) • https://imagesharer.xyz/
References	• https://cwe.mitre.org/data/definitions/284.html • https://cwe.mitre.org/data/definitions/359.html • https://docs.aws.amazon.com/AmazonS3/latest/userguide/access-control-block-public-access.html

Overview

The S3 bucket that stores every uploaded image for Imager is configured so that any person on the internet, with no credentials, can list every file and download any image - including images users explicitly marked as "Private". The platform's privacy promise does not match reality: there is effectively no such thing as a private image on Imager.

Details

Two anonymous (unauthenticated) HTTP requests were made directly to the bucket:

1. `GET https://filesharer-research.s3.ap-southeast-2.amazonaws.com/?list-type=2` -> HTTP 200. Response is a standard `ListBucketResult` XML containing all 49 object keys at the time of testing. Each key follows the pattern `users/<user_uuid>/<image_uuid>.<ext>`, revealing all user IDs and all image IDs stored in the system.
2. `GET https://filesharer-research.s3.ap-southeast-2.amazonaws.com/users/0c3f8747-73f2-4f1f-b453-010f5b7fa950/925e5bda-6b04-4dfb-9b92-d557da791d57.jpg` -> HTTP 200, Content-Type image/jpeg, 628 bytes of valid JPEG data. This object was a PRIVATE image uploaded by test Account B during authorization-flow testing; it was fetched without any credentials, cookies, or signed URL.

Additional observations.

- 22 distinct `user_<uuid>/` directories were enumerated with a single request.
- Several directory names contained invalid characters (`"`, `'`, empty string, and arbitrary text) — a direct consequence of the upload mass-assignment IDOR (separate finding), which has clearly been exercised in the wild and polluted the bucket keyspace.
- Anonymous PUT and DELETE were correctly rejected (HTTP 403); the bucket is READ-ONLY to the public, not writable.

Business impact. Full confidentiality loss for every image ever uploaded. Private images are not private. Privacy / Notifiable Data Breach exposure under the Australian Privacy Act 1988 if any identifying content exists in user images. Legal and reputational risk: Imager's "your images, your

control" tagline is directly contradicted by this misconfiguration. Attackers can enumerate all user UUIDs and correlate them with the public-facing `/images/<uuid>` URL space to systematically harvest content. Combined with the IDOR findings and the leaked AWS root key, an attacker has multiple independent paths to the same data.

Evidence.

- `evidence/p2-s3-and-enum/s3list.xml`, `s3all.xml` (anonymous ListBucket)
- `evidence/p2-s3-and-enum/retry/list.xml`, `obj-*.bin` (anonymous image downloads)
- `evidence/p2-final/p1-fetch.*` (private image fetched anonymously)

Recommendation

Immediate:

1. Enable AWS "Block Public Access" at both the account and bucket level on `filesharer-research` (ap-southeast-2): S3 Console -> Buckets -> filesharer-research -> Permissions -> Block public access -> enable all 4 settings.
2. Review the bucket policy and object ACLs; remove any `Principal: "*"` or `AllUsers`/ `AuthenticatedUsers` grants.
3. Serve images via a signed URL mechanism that requires an authentication check inside Imager's application (the app already generates pre-signed URLs — it just needs the bucket to default-deny unsigned requests).
4. Rotate any sensitive data that may have been exposed; notify affected users as required by the Notifiable Data Breaches scheme if any PII-bearing images are in the bucket.

Short term:

5. Shorten the pre-signed URL `X-Amz-Expires` from 604800 seconds (7 days) to the minimum operationally required (5-15 minutes).
6. Add CloudTrail and GuardDuty monitoring for S3 bucket activity. Alert on anonymous requests and on mass `GetObject` patterns.
7. Enable S3 Server Access Logging (or CloudTrail data events) and review the last 90 days for signs that external parties have already enumerated the bucket.
8. Decide and apply a key-naming policy that does NOT embed trust in client-provided fields (`user_id`). Prefer server-generated, random keys stored and resolved via the database.

H1: Mass Assignment / IDOR on Image Upload (Attacker Uploads Images to Any Victim Account)	
Score	7.6 (High)
Vector string	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:H/A:L
Target	POST https://imagesharer.xyz/images/upload
References	• https://cwe.mitre.org/data/definitions/639.html • https://cwe.mitre.org/data/definitions/915.html • https://owasp.org/API-Security/editions/2023/en/0xa3-broken-object-property-level-authorization/

Overview

When a signed-in Imager user uploads a picture, the upload form includes a hidden `user_id` field that identifies who owns the image. The server trusts whatever value the browser sends in that field instead of using the authenticated session. Any logged-in user can paste another member's user ID into the request and the picture is saved as if the other member uploaded it - appearing on their dashboard, counted against their quota, and attributed to them in any audit log.

Details

Upload page HTML (`/images/upload`) includes:

```
<input name="user_id" type="hidden" value="<caller's own UUID>">
<input name="file" type="file" required>
<input name="is_public" type="radio" ...>
```

Reproduction (verified):

1. Account A (UUID `c9178440-59d6-4715-b8dc-782c592c5885`) was authenticated via `PHPSESSID`.
2. Multipart `POST /images/upload` with field `user_id` = Account B's UUID (`0c3f8747-73f2-4f1f-b453-010f5b7fa950`).
3. Server returned HTTP 302 `Location: /images/ed9b9ab1-b69c-40a7-a2eb-3582f889a67c`.
4. Account B's dashboard (fetched independently) now shows image `ed9b9ab1-b69c-40a7-a2eb-3582f889a67c` as owned by B, even though B never uploaded it.

Full raw request/response captured in `evidence/p2-uploads/idor/`.

Business impact. An attacker can plant content on any victim's account. For an image-sharing platform this is severe: an attacker can plant unlawful or defamatory images under a known user's name, exposing that user (and Imagur) to legal, reputational, and law-enforcement consequences. In the worst case, Imagur itself may be served with takedown notices for content it believes a specific

user chose to upload. The defect can also be used to exhaust a user's storage quota (per-user DoS) or spam a victim's gallery. The forensic trail is destroyed: logs will say "User B uploaded X" when in fact User A did.

Evidence.

- `evidence/p2-uploads/idor/baseline.body`, `baseline.headers`
- `evidence/p2-uploads/idor/idor.body`, `idor.headers`
- `evidence/p2-uploads/idor/dash-a.html`, `dash-b.html`

Recommendation

1. Remove the hidden `user_id` field from the HTML form entirely — the browser cannot be trusted to tell the server which user is logged in.
2. In the upload handler, derive the owning `user_id` SOLELY from the authenticated session (e.g., `$_SESSION['user_id']`). Discard any `user_id` value that arrives in the POST body.
3. Add an integration test: as user A, submit a crafted multipart POST with `user_id` = user B. Assert the stored row's owner matches A.
4. Review every other endpoint that takes an entity ID in the body for the same pattern (`POST /profile/avatar`, `POST /logout`, `POST /images/{id}/visibility` — see separate finding).
5. Consider migrating to a framework-level pattern (e.g., Laravel's `$request->user()` / policies) that makes it hard to accidentally trust client-supplied owner IDs.

H2: Broken Access Control on Image Visibility Toggle (IDOR - Attacker Makes a Victim's Private Image Public)	
Score	7.1 (High)
Vector string	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:L/A:N
Target	POST https://imagesharer.xyz/images/<uuid>/visibility
References	• https://cwe.mitre.org/data/definitions/639.html • https://cwe.mitre.org/data/definitions/285.html • https://owasp.org/Top10/A01_2021-Broken_Access_Control/

Overview

Any logged-in user who knows (or guesses from an email, dashboard link, or shared URL) another user's image ID can switch that image from "private" to "public" without owning it. Private images are therefore not reliably private: a single HTTP request from another member exposes them to the open internet.

Details

Reproduction (verified):

1. Account B uploaded a PRIVATE image (ID `925e5bda-6b04-4dfb-9b92-d557da791d57`).
2. Account A sent `POST /images/925e5bda-6b04-4dfb-9b92-d557da791d57/visibility` with body `is_public=true`, authenticated as A.
3. Server responded HTTP 302 to the image detail page (success flow).
4. Account B's dashboard subsequently displayed the image as Public without B taking any action.

The server does not check that the authenticated user owns the image before changing a sensitive property. The same pattern on the delete endpoint was observed to be correctly gated (image remained after A's attempt), confirming an authorization check exists in the code base — it simply was not applied here.

Business impact. Direct violation of the user's privacy expectation ("Private = only you"). A malicious or compromised account can run through every image UUID it learns about (shared links, scraping public pages, leaked log data) and silently flip them to public. The now-public image is retrievable via its `/images/<uuid>` page AND via a long-lived S3 pre-signed URL (see separate finding). Combined with the image upload IDOR, an attacker can plant an image on a victim's account AND make it public, then advertise the URL — a credible content-planting plus exposure attack.

Evidence.

- `evidence/p2-authz/authz-tests.log`
- `evidence/p2-authz/s3.headers`, `s3.body`

Recommendation

1. In the visibility handler, SELECT the image row and verify `image.owner_id == $_SESSION['user_id']` BEFORE applying the update. Return HTTP 403/404 otherwise.
2. Apply the same explicit ownership check to every `/images/<uuid>/*` endpoint. Treat the per-image delete endpoint as the reference implementation and extend the same pattern.
3. Add server-side logging and alerting when an ownership check fails. Repeated failures from one session are a strong IDOR-probing signal.
4. Add a regression test: user A tries to change visibility of a UUID owned by B — expect 403/404 and no DB mutation.

H3: Production Application Also Exposed Over Plaintext HTTP on Port 8080	
Score	7.1 (High)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:L/A:N
Target	http://51.161.131.44:8080/
References	• https://cwe.mitre.org/data/definitions/319.html • https://datatracker.ietf.org/doc/html/rfc7258

Overview

The Imager application is served not only over HTTPS on port 443 (via imagesharer.xyz) but also over PLAINTEXT HTTP on port 8080 directly on the server's IP. Anyone visiting http://51.161.131.44:8080/ gets the real, logged-in, interactive production application without TLS. If a user signs in that way (or is tricked into clicking such a link), their email, password, session cookie, and uploaded images traverse the network in the clear and can be captured by anyone on the same network path.

Details

- `curl http://51.161.131.44:8080/login` returns the identical "Log in — Imager" HTML as the HTTPS site (byte-identical body).
- The same `/js/awsvalid.js` (containing the leaked AWS root credential) is served from this port.
- The server does NOT redirect port 8080 to HTTPS and does NOT reject plaintext requests.
- There is no vhost gating (Host header does not affect behavior).
- Nothing on the production site advertises port 8080, so it is unlikely to be used by legitimate users — but it is reachable by any scanner on the internet and by any attacker who crafts a link.

Business impact. Session hijacking, credential theft, and image theft for any user who authenticates over port 8080. Phishing lure: an attacker who sends `http://51.161.131.44:8080/login?return=...` can intercept credentials in a controlled MITM/relay setup. Weakens the security story: Imager appears HTTPS-only to the end user, but the backend is reachable in cleartext.

Evidence. `evidence/p3-infra/other-ports/*` and `port8080/*` (banner, title, awsvalid.js, side-by-side hashing)

Recommendation

1. Close port 8080 at the OS or OVH firewall. If the port is only needed for internal process-manager health checks, bind the Apache listener to `127.0.0.1` instead of `0.0.0.0`.
2. If port 8080 must remain publicly reachable (e.g., for a load balancer that cannot be reconfigured right now), make it serve ONLY a permanent HTTPS redirect to `https://imagesharer.xyz/`, and set

`Strict-Transport-Security` on the HTTPS response (see Headers finding) so browsers remember HTTPS.

3. Audit the server for any other Apache/nginx vhost listening on non-443 ports; apply the same policy.

M1: No Rate Limiting on /login (Enables Credential Brute Force)	
Score	6.5 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N
Target	POST https://imagesharer.xyz/login
References	• https://cwe.mitre.org/data/definitions/307.html • https://owasp.org/www-community/attacks/Brute_force_attack

Overview

Failed login attempts are not rate-limited. Five consecutive wrong-password attempts in rapid succession all returned the same `302 -> /login` response with no 429, no CAPTCHA, no account lockout, no exponential backoff. An attacker can therefore attempt thousands of passwords per second against a known email without obstruction.

Details

Five rapid POSTs to `/login` with the real test email and five different wrong passwords all returned HTTP 302 immediately (sub-200 ms each). No throttling observed.

Business impact. Online password guessing against known-good emails (which the registration-enumeration finding makes easy to collect) is feasible. Combined with the weak-password-policy finding (users may have 1-character passwords), compromise is realistic. Credential stuffing with leaked-password datasets costs essentially nothing to run.

Evidence. `evidence/p2-final/final-tests.log` (R-1 section)

Recommendation

1. Rate-limit `/login` by (IP + email) key: for example, after 5 failures in 15 minutes, respond with HTTP 429 and a `Retry-After` header for 15 minutes.
2. Add exponential backoff / login-captcha after the second failure within a short window.
3. Alert on sustained high failure rates per IP range for operational awareness.
4. Prefer a WebAuthn / passkey option so password brute force is simply not applicable.

M2: PostgreSQL Database Server Exposed to the Public Internet	
Score	6.5 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:L
Target	51.161.131.44:5432/tcp
References	• https://cwe.mitre.org/data/definitions/668.html • https://www.postgresql.org/docs/current/auth-pg-hba-conf.html • https://cheatsheetseries.owasp.org/cheatsheets/Network_Segmentation_Cheat_Sheet.html

Overview

The database powering imagesharer.xyz is reachable from any address on the public internet. The application and the database are co-located on the same server, so no external party has a legitimate reason to connect to port 5432. The open port creates a direct path for attackers to attempt credential guessing, exploit PostgreSQL vulnerabilities, and (if they obtain credentials) steal, modify, or destroy all image and user data.

Details

A targeted service probe to 51.161.131.44 TCP/5432 returned:

```
PORT     STATE SERVICE    VERSION
5432/tcp open  postgresql PostgreSQL DB
Protocol: 3.0 (confirmed via PostgreSQL error response)
TLS on 5432: not configured
```

The service accepts arbitrary TCP connections from the internet. No authentication attempt was made (out of scope per rules of engagement), but the exposure alone materially increases risk:

- Any attacker, including unattended internet-wide scanners, can connect and attempt to authenticate.
- Credential guessing against `pg_hba.conf`'s `md5` / `scram-sha-256` is possible at scale.
- Historical PostgreSQL CVEs (RCE, authentication bypass, privilege escalation) are reachable before they can be patched.
- Data is transmitted in cleartext because TLS is not enforced on the port.

Business impact. A single stolen or guessed database password equals full loss of user data (images, usernames, password hashes, email addresses, sessions). Compliance implications under the Australian Privacy Act / Notifiable Data Breaches scheme if real user data is accessed.

Evidence. `evidence/p0-postgres/nmap-5432.txt`

Recommendation

Immediate:

1. Restrict TCP/5432 at the firewall. Simplest fixes on OVH:
   - Edit `postgresql.conf`: `listen_addresses = 'localhost'` and restart PostgreSQL. The web app on the same server will still connect via 127.0.0.1.
   - If a separate DB host is needed, allow 5432 only from the application's IP in `pg_hba.conf` AND restrict with ufw/iptables / OVH security group.
2. Verify no PostgreSQL password uses `postgres`, `password`, `admin`, or an empty value. Rotate the application's DB user password to a long, randomly generated secret.

Short term:

3. Enable TLS on PostgreSQL (`ssl = on`) with a valid certificate, and require `hostssl` entries in `pg_hba.conf`.
4. Enable `pg_hba.conf` method `scram-sha-256` (not `md5`) for all users.
5. Keep PostgreSQL packages patched (currently ambiguous on-wire version — ensure latest 16.x LTS).
6. Add fail2ban or equivalent to lock out hosts after repeated failed PostgreSQL logins.

M3: Private Images Expose Long-Lived Pre-signed URLs That Bypass "Private" Status	
Score	6.5 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N
Target	GET https://imagesharer.xyz/images/<uuid> (owner view for PRIVATE images)
References	• https://cwe.mitre.org/data/definitions/359.html • https://cwe.mitre.org/data/definitions/613.html • https://docs.aws.amazon.com/AmazonS3/latest/userguide/ShareObjectPreSignedURL.html

Overview

When the owner of a PRIVATE image opens its detail page, Imager inserts an AWS S3 pre-signed URL into the HTML - including a "Direct link" field users may copy. This URL is a bearer token that works for anyone, from anywhere, for 7 full days, regardless of whether the image remains private. If a user shares the URL once (messaging, screenshot, log file, help ticket, browser history export), the "private" image is effectively public for a week.

Details

Reproduction (verified):

1. Account A uploaded a PRIVATE image (ID `4cbfdbb9-52f1-42e4-ba95-33b32e40ec2b`).
2. Account A opened `/images/4cbfdbb9-52f1-42e4-ba95-33b32e40ec2b`. The HTML included:
   - An `<img src="...X-Amz-Signature=...X-Amz-Expires=604800...">` element.
   - A "Direct link" `<input type="text" readonly>` containing the same signed URL for easy copying.
3. The signed URL was taken out-of-band and fetched with no session cookie, no referrer, from an unrelated client -> HTTP 200, image/jpeg, full image bytes.
4. The `X-Amz-Expires=604800` parameter indicates the URL is valid for 7 days regardless of subsequent visibility changes on the Imager side.

Note: This finding is independent of the public-bucket finding. Even if the bucket is correctly locked down, pre-signed URLs remain valid, by design of AWS, until expiry.

Business impact. Users have a strong, implicit expectation that "Private" means "nobody else can view this". That contract is broken. A single accidental URL share leaks the image for 7 days, and the URL cannot be revoked individually. Combined with the public bucket, the attack surface is large; fixing one does not fix the other.

Evidence. `evidence/p2-final/p1-up.*`, `p1-detail.html`, `p1-fetch.*`

Recommendation

1. Do NOT render pre-signed URLs for private images on the owner's detail page. Instead, serve a server-mediated endpoint such as `GET /images/<uuid>/raw`, which checks session + ownership + current visibility on every request and streams the file.
2. If pre-signed URLs are retained for public images, cap `X-Amz-Expires` to the minimum operationally required (default 300 seconds). Never 7 days.
3. If a user toggles a public image to private, immediately rotate / overwrite the object (or set a new object key) so any outstanding pre-signed URL stops working.
4. Add a user-visible "Rotate link" button so owners can invalidate the current direct link on demand.

M4: Weak Password Policy - Single-Character Passwords Accepted	
Score	6.5 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N
Target	POST https://imagesharer.xyz/register
References	• https://cwe.mitre.org/data/definitions/521.html • https://pages.nist.gov/800-63-3/sp800-63b.html

Overview

The registration form claims to require passwords of 8 characters or more, but this check is enforced only in the browser (HTML `minlength=8`). A direct HTTP POST with a one-character password is accepted and the account is created with that one-character password. Users following the UI get a reasonable policy; attackers who send raw requests get none.

Details

Observed:

- Legitimate flow via browser: `minlength=8` enforced client-side; UX gated.
- Direct POST: `curl -X POST -d 'email=weakpw-test@test.local&password=a&confirm=a' https://imagesharer.xyz/register` -> HTTP 302 (success response).

The companion issue was also tested: mismatched `password` and `confirm` fields are NOT enforced server-side either; the password is set to whatever `password` contains and `confirm` is cosmetic.

Business impact. Any user who registers via API, test harness, mobile client, or scripted sign-up can establish an account whose password is trivial to guess. When combined with the lack of rate limiting on `/login` (separate finding) and the user-enumeration on `/register` (separate finding), credential stuffing / targeted password guessing becomes very cheap.

Evidence.

- `evidence/p2-misc/p4.body`, `p4.headers` (1-char password accepted)
- `evidence/p2-misc/p5.body`, `p5.headers` (confirm mismatch accepted)

Recommendation

1. Enforce the password policy server-side (minimum 8 characters, mix of character classes OR length >= 12 — follow NIST 800-63B).
2. Server-side validate `password === confirm` and reject with HTTP 400 otherwise.

3. Check new passwords against a well-known breached-password list (e.g., HaveIBeenPwned k-anonymity API).
4. Return explicit, actionable validation errors so legitimate users can self-correct.

M5: Open DNS Resolver on Port 53 (Potential DDoS Amplification Reflector)	
Score	5.8 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:N/I:N/A:L
Target	51.161.131.44:53 (udp/tcp)
References	• https://www.cisa.gov/news-events/alerts/2013/03/29/dns-amplification-attacks • https://www.cloudflare.com/learning/dns/dns-amplification-attack/ • https://openresolverproject.org/

Overview

The server is running an open DNS resolver that recursively answers queries from anyone on the internet. The server returned a correct answer when asked to resolve `example.com` from an unrelated source address. Open resolvers are routinely abused as reflectors in DDoS amplification attacks: an attacker spoofs a victim's IP, sends a small query, and the resolver sends a large reply to the victim. Imager's server would be the unwitting attacker in such an incident, leading to IP reputation damage, upstream provider complaints, and potential service degradation while the DDoS traffic is sourced from this host.

Details

- `dig @51.161.131.44 example.com +short` returned Cloudflare IPs for example.com, confirming full recursive resolution for an off-domain query.
- `version.bind CHAOS TXT` and `hostname.bind CHAOS TXT` were blocked / returned no useful banner — good.
- Direct impact to Imagur is modest, but reputational and operational exposure is real: upstream (OVH) may null-route the host under abuse complaint; the server contributes to internet-wide DDoS.

Business impact. Potential upstream null-route by OVH if the server is reported in a DDoS incident. Resource consumption during amplification abuse (bandwidth, CPU). Reputational: Imagur's IP may land on abuse databases, affecting outbound mail or API callbacks later.

Evidence. `evidence/p3-infra/other-ports/probe-other.log`

Recommendation

1. If this server is not supposed to be a DNS server, STOP the resolver daemon (bind9/unbound/dnsmasq/systemd-resolved) or bind it to `127.0.0.1` only.

2. If it must run DNS, restrict recursion with ACLs:
   - BIND: `allow-recursion { 127.0.0.1; <trusted-ranges>; };`
   - Unbound: `access-control: 0.0.0.0/0 refuse` then allow specific ranges.
3. Add rate limiting (`rate-limit { responses-per-second 5; };` in BIND) as defense-in-depth.
4. Verify the fix publicly via https://openresolverproject.org/ or `dig @51.161.131.44 example.com` from an off-net client.

M6: Dangling / Misconfigured CloudFront Alias on resources.imagesharer.xyz	
Score	5.4 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N
Target	• resources.imagesharer.xyz • d273uevi0662l0.cloudfront.net
References	• https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/CNAMEs.html • https://cwe.mitre.org/data/definitions/1104.html • https://cwe.mitre.org/data/definitions/295.html

Overview

Imagur's DNS points `resources.imagesharer.xyz` to an AWS CloudFront distribution that is not configured to accept that hostname. Browsers requesting the URL see a security warning (the TLS certificate does not cover the hostname), and the distribution returns HTTP 403 for every request. The intended behavior (serving the site's favicon from `https://resources.imagesharer.xyz/icon.ico`) is broken. The state also leaves a smaller risk of future takeover if the CloudFront distribution is ever deleted while the DNS record remains.

Details

Observed:

- DNS: `resources.imagesharer.xyz CNAME d273uevi0662l0.cloudfront.net`
- TLS (SNI `resources.imagesharer.xyz`): certificate `CN=*.cloudfront.net`, SANs `*.cloudfront.net`, `cloudfront.net` only — DOES NOT include `resources.imagesharer.xyz`, so every browser request raises a certificate error.
- HTTP: 403 Forbidden on both `/` and `/icon.ico` with headers `Server: CloudFront` and `X-Cache: Error from cloudfront`, confirming the distribution exists but does not have `resources.imagesharer.xyz` listed as an alternate domain name (CNAME alias).
- Direct access to `d273uevi0662l0.cloudfront.net` reveals an origin of AmazonS3 (empty 404), meaning the origin bucket is either empty or unconfigured.
- The main login page references this hostname (favicon), so any user who views page source or any future dependency on this host will encounter broken resources and TLS warnings — degrading trust and triggering browser phishing-detection heuristics on some clients.

Takeover risk. If the CloudFront distribution is ever deleted (common during cost cleanup) and the DNS CNAME is not removed, a different AWS customer could set up a CloudFront alias for `resources.imagesharer.xyz` and serve content from it. The present risk is low (the distribution is active), but it will materialize the moment the distribution is deleted.

Business impact. Broken intended functionality (favicon / static assets). Loss of user trust if any URL is shared publicly (browser certificate warning page). Latent subdomain-takeover risk if the distribution is ever deprovisioned.

Evidence. `evidence/p0-cloudfront/cloudfront-check.txt`

Recommendation

Choose ONE:

A) If the CDN subdomain is still needed:

1. In the CloudFront distribution (`d273uevi0662l0.cloudfront.net`) -> Settings -> Alternate domain names (CNAMEs), add `resources.imagesharer.xyz`.
2. Request a public ACM certificate (us-east-1) for `resources.imagesharer.xyz` and attach it to the distribution.
3. Confirm the S3 origin bucket contains the expected content (icon.ico, static assets) and that Origin Access Control is correctly configured.

B) If the CDN subdomain is NOT needed:

1. Delete the `resources.imagesharer.xyz` CNAME record at the DNS provider (Spaceship).
2. Remove the (commented) HTML reference in the login page.

C) General:

3. Document a rule in deployment / off-boarding checklists: never delete a CloudFront distribution or S3 bucket without first removing or re-pointing the DNS record that points to it.

M7: Unhandled Server Error on Empty user_id Parameter (Denial via Malformed Input)	
Score	5.4 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:L
Target	POST https://imagesharer.xyz/images/upload
References	• https://cwe.mitre.org/data/definitions/209.html • https://cwe.mitre.org/data/definitions/20.html

Overview

When an authenticated user submits an image upload with an empty `user_id` field, the server returns an HTTP 500 `INTERNAL_ERROR` instead of a validated HTTP 400 Bad Request. This indicates a missing input-validation path and leaks information about internal state.

Details

Request: `POST /images/upload` with `user_id=&file=<jpeg>&is_public=false`, cookie present. **Response:** HTTP 500 with an `INTERNAL_ERROR` body.

While minor on its own, this is diagnostic evidence that the upload handler trusts client input blindly (consistent with the mass-assignment IDOR finding) and lacks defensive validation.

Business impact. Minor information disclosure (presence of an unhandled branch). Each triggering request wastes server resources.

Evidence. `evidence/p2-final/u2.h`, `u2.b`

Recommendation

1. Remove the `user_id` form field entirely (see the Mass Assignment finding). Once removed, this error path no longer applies.
2. Add a generic request-validation layer that returns HTTP 400 for any missing/invalid required fields and never returns 500 for user-controllable input.
3. Ensure production error responses do not disclose stack traces, class names, or framework internals.

M8: User-Enumeration via Registration Endpoint	
Score	5.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Target	POST https://imagesharer.xyz/register
References	• https://cwe.mitre.org/data/definitions/204.html • https://owasp.org/www-community/vulnerabilities/Information_exposure_through_differential_error_messages

Overview

When a visitor attempts to register an email that is already in use, Imager replies with the explicit message "An account with that email already exists.". Attackers can therefore test any email against the site and learn whether that person has an Imager account. (For comparison, the `/login` endpoint correctly returns the same message for both unknown-email and wrong-password cases - this is good; `/register` must be aligned with that behaviour.)

Details

Verified with explicit POST + follow-up GET of `/register` as the same session. Known-registered email yields `"An account with that email already exists."`. Unknown emails produce the normal signup success flow. The login endpoint returns an identical `"Invalid email or password."` for both wrong-password and unknown-email (good — do not change this).

Business impact. Enables attackers to confirm whether specific individuals have accounts, which supports targeted phishing, social engineering, and credential stuffing campaigns. Enables bulk account mapping from a list of email addresses.

Evidence.

- `evidence/p2-s3-and-enum/retry/getC.html` (quoted error)
- `evidence/p2-s3-and-enum/retry/getA.html`, `getB.html` (login — identical)

Recommendation

1. On `/register`, do NOT reveal whether the email is already registered. Return a generic success message and send an email to the address: either (a) a verification email if new, or (b) a "someone tried to re-register this address" notice if already present. The flow is unchanged for legitimate owners; attackers get no signal.
2. Combine with CAPTCHA + registration rate limiting (per IP, per email) to raise the cost of bulk enumeration.

M9: Missing HTTP Security Headers (HSTS, CSP) and Email Anti-Spoofing (SPF, DMARC)	
Score	4.2 (Medium)
Vector string	CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:L/A:N
Target	• https://imagesharer.xyz/ (all HTTP responses) • DNS zone imagesharer.xyz
References	• https://cheatsheetseries.owasp.org/cheatsheets/HTTP_Headers_Cheat_Sheet.html • https://datatracker.ietf.org/doc/html/rfc6797 • https://content-security-policy.com/ • https://datatracker.ietf.org/doc/html/rfc7489

Overview

Several low-cost hardening headers and DNS records are missing. Each on its own is minor, but together they weaken defense-in-depth against TLS downgrade attacks, cross-site scripting, and email spoofing.

Details

Observed on `https://imagesharer.xyz/`:

- `Strict-Transport-Security`: NOT SET -> browsers can be downgraded to HTTP on a hostile network before the first HTTPS visit of a session.
- `Content-Security-Policy`: NOT SET -> no second-layer defense against XSS; inline scripts, inline event handlers, and arbitrary CDN origins are permitted.
- `Permissions-Policy`: NOT SET -> no restrictions on camera/microphone/geolocation APIs if ever abused by a stored XSS.
- `Cross-Origin-Opener-Policy` / `Cross-Origin-Embedder-Policy`: NOT SET.

Observed in DNS for `imagesharer.xyz`:

- `MX` records: NONE present.
- `SPF` (`TXT v=spf1`): NOT present.
- `DMARC` (`TXT _dmarc.imagesharer.xyz`): NOT present.
- `CAA`: NOT present.

The domain can therefore be freely spoofed as a sender in email, since there is no SPF `-all` / `v=spf1 -all` policy and no DMARC `reject` policy. Attackers can send mail "from" `help@imagesharer.xyz` with high deliverability.

Business impact. Easier for attackers to run convincing phishing / password-reset lures that appear to come from Imager. No protection against TLS downgrade for first-time visitors on hostile networks

(cafe wifi, hotels, airport). No browser-level containment should a stored XSS appear anywhere in the application.

Evidence.

- `evidence/p1-discovery/root.headers.txt`
- Enumeration YAML (`dns_infrastructure` / `mail_infrastructure` sections)

Recommendation

1. Add response headers at the nginx layer:
   - `Strict-Transport-Security: max-age=63072000; includeSubDomains; preload`
   - `Content-Security-Policy: default-src 'self'; img-src 'self' https://filesharer-research.s3.ap-southeast-2.amazonaws.com data:; style-src 'self' 'unsafe-inline'; script-src 'self'; frame-ancestors 'none'; form-action 'self'; base-uri 'self'`
   - `Permissions-Policy: camera=(), microphone=(), geolocation=()`
2. Add DNS records at Spaceship DNS:
   - `imagesharer.xyz. TXT "v=spf1 -all"` (or `-all` plus the real sending host if mail will be sent).
   - `_dmarc.imagesharer.xyz. TXT "v=DMARC1; p=reject; rua=mailto:security@imagesharer.xyz"`
   - `imagesharer.xyz. CAA 0 issue "letsencrypt.org"` to pin which CAs may issue certificates.
3. After HSTS has been live for >= 6 months with no issues, submit the domain to the HSTS preload list.

I1: SSH (OpenSSH 10.0p2) Exposed to the Public Internet	
Score	0.0 (Info)
Vector string	N/A
Target	51.161.131.44:22/tcp
References	https://cheatsheetseries.owasp.org/cheatsheets/SSH_Hardening_Cheat_Sheet.html

Overview

SSH is open to the entire internet. This is extremely common for small cloud VPSes but still represents an unnecessary attack surface. There is nothing immediately exploitable in the OpenSSH banner (10.0p2 is current), but any future SSH CVE or any weak password / credential leak becomes directly exploitable from anywhere.

Details

Nmap banner: `22/tcp open ssh OpenSSH 10.0p2 Debian 7+deb13u4 (protocol 2.0)`. No evidence of banner suppression; no attempt was made (nor permitted) to authenticate.

Recommendation

Defense in depth, not urgent:

1. Restrict SSH to the administrator's IP / VPN at the OVH firewall or via `AllowUsers` / `Match Address` in `sshd_config`.
2. Disable password authentication (`PasswordAuthentication no`) and allow only public-key auth.
3. Disable root login (`PermitRootLogin no` or `prohibit-password`).
4. Install `fail2ban` and set `MaxAuthTries 3` to deter online guessing.
5. Keep OpenSSH patched promptly (automated apt-upgrade or unattended-upgrades).

5 Disclaimer

The purpose of this research project is to help evaluating the seamlessness, effectiveness and cost-benefit ratio of autonomously performing vulnerabilty assessment using agentic AI. I do not take any responsibility upon any misuse or abuse of such tool where it can face significant legal actions.

A Appendix

A.1 Static Appendix Section

A.2 Methodology

The engagement followed ClawSec's standard workflow: INTAKE -> AUTHORIZED -> ENUMERATED -> ASSESSING -> VALIDATING -> REPORTING.

Asset discovery. DNS, WHOIS, TLS SAN inspection, Certificate Transparency logs, Shodan, HackerTarget, and the Wayback Machine were used to identify assets owned by the business. The confirmed scope was validated with the owner before any active testing began.

Web application testing. Burp-style HTTP probes using two engagement-only test accounts (clawsec-pt-a / clawsec-pt-b). Test classes exercised: unauthenticated SQL injection, authentication bypass, session fixation, mass-assignment / IDOR, broken object-level access control, file upload abuse (MIME + extension + polyglot + path traversal), stored XSS via filename, user enumeration via login/register, rate limiting, HTTP method abuse, and pre-signed URL inspection.

Infrastructure testing. `nmap` top-1000 TCP service enumeration at gentle timing (T3); TLS version and cipher enumeration via `openssl s_client`; PostgreSQL banner probe (no authentication attempts); DNS open-resolver check with `dig`.

AWS testing. A single read-only `aws sts get-caller-identity` call was made using the publicly leaked access key, solely to confirm ownership. No listing, no mutation, and no further API calls were performed with the leaked credential.

S3 testing. Anonymous HEAD, GET, and ListBucket requests were made against the bucket; anonymous PUT and DELETE write-probes were sent and both were correctly denied.

Standards referenced: OWASP Top 10 2021, OWASP ASVS, NIST SP 800-63B, AWS Security Best Practices.

A.3 Remediation Summary (Executive)

The following prioritized actions address the issues identified during the engagement.

Within hours (do first):

1. Rotate the exposed AWS root access key and remove it from the public JavaScript file.
2. Enable "Block Public Access" on the `filesharer-research` S3 bucket.

Within days (before any further user growth):

3. Fix the two broken access-control issues on the image upload and image visibility endpoints so a user cannot act on behalf of another user.
4. Replace long-lived (7-day) pre-signed image URLs with short-lived server-mediated access.
5. Firewall PostgreSQL (port 5432) and close plaintext HTTP on port 8080.
6. Disable the open DNS resolver on port 53 or restrict it to trusted ranges.

Within the next sprint:

7. Enforce the password policy on the server, not only in the browser.
8. Add rate limiting to the login endpoint.
9. Make the registration endpoint stop revealing whether an email is already registered.
10. Add HSTS, CSP, and Permissions-Policy response headers; add SPF, DMARC, and CAA DNS records.
11. Harden SSH exposure (key-only authentication, restrict source IPs).

Longer term:

12. Move all secrets to a secret manager and never embed them in client-side code.
13. Add CI/CD secret-scanning (gitleaks / trufflehog) to prevent future credential leaks.
14. Enable AWS GuardDuty and CloudTrail data events for S3; alert on anonymous requests.